

Summary Sheet

Team Control Number: #M2625838

We study *Dancing with the Stars* (DWTS) as an inverse problem: judges' scores and weekly eliminations are observed across 34 seasons[1], but audience vote totals are withheld. The key unknown is each active contestant's weekly fan vote share, which drives eliminations under different aggregation rules.

What we built

- A reproducible season-week-contestant panel that standardizes active sets, elimination labels, and judge-score aggregates under irregular show formats (no-elimination weeks, multi-elimination weeks, varying numbers of judges, and post-exit “zero” encodings).
- A convex inference model (Task A) that reconstructs latent fan vote shares consistent with observed eliminations, and an identifiability measure (KPI2) quantifying how uniquely votes are determined each week.
- A replay engine (Tasks B/C) that re-simulates every season under percent-based versus rank-based aggregation (optionally with a bottom-two judges' save), enabling controlled counterfactual comparisons and controversy case studies.
- Mixed-effects attribution models (Task D1) separating drivers of judges' scores and fan support, and a simple, interpretable new voting rule (Task D2) stress-tested by replay and uncertainty perturbations.

Sanity-check evidence (data integrity)

From the weekly panel we define *eligible* season-weeks (including multi-elimination weeks evaluated by bottom- k set match). Under a BL-0 baseline (judge-only with a uniform fan proxy), the elimination hit-rate is 0.364 (rank aggregation) and 0.375 (percent aggregation) across 264 eligible weeks, and the two schemes disagree on the predicted eliminated set in 3.8% of eligible weeks (FlipRate = 0.038). These rule divergences are consistent with classical observations in voting and aggregation theory.[2] These metrics validate consistent labeling and replay mechanics and serve as a baseline—not the final inferred-vote results. When counterfactual replays require post-exit inputs, we use a conservative, minimal-information imputation only to complete the replay, and we do not interpret these imputed segments as forecasting contestant performance.

Key deliverables for decision-making

- Season-by-season comparisons of how often rank vs. percent rules change eliminations and winners, and when judges' save overturns vote-driven outcomes.
- Uncertainty reports showing which weeks/contestants are non-identifiable from observed eliminations alone.
- A recommended voting system with two intuitive control knobs (judge-fan balance and popularity compression), selected via multi-metric stress tests to improve stability without sacrificing interpretability.

Management recommendation

Pilot the proposed rule in a one-season “shadow mode” (computed in parallel with the broadcast rule) and adopt a parameter region that preserves replay fidelity on eligible weeks while improving stability under vote-uncertainty perturbations.

Contents

1	Introduction	2
2	Problem Decomposition and Analysis	2
2.1	Problem Decomposition	2
2.2	Key Assumptions	3
2.3	Key Notations	3
2.4	Tasks and Evaluation KPIs	4
3	Data Processing and Analysis	5
3.1	Data Cleaning Principles and Validation	5
3.1.1	Cleaning Principles	5
3.1.2	Cleaning Principles	6
3.1.3	Data Cleaning Outcomes and Validation	6
3.2	Panel Construction	7
3.3	Empirical Findings from EDA (only what feeds models)	7
3.3.1	Data Visualization	7
3.3.2	Key Empirical Findings	7
3.4	Feature Engineering	8
3.5	Synthesis of Findings	8
4	Task A: Fan Vote Inference Model	8
4.1	Overview and Modeling Philosophy	8
4.2	Main Inference Model	9
4.3	Uncertainty Quantification and Sensitivity Analysis	10
4.3.1	KPI2: Vote Identifiability (interval tightness)	10
4.4	Hyperparameters, tuning, and implementation details	12
5	Task B & C: Rule Reply, Volting System, and Counterfactuals	13
5.1	Rank vs. Percent Across Seasons	13
5.1.1	Replay protocol and eligibility	13
5.1.2	Cross-season summary metrics	14
5.1.3	Era cutoff assumption and stress-test plan	14
5.2	Controversial Contestants: Case Studies	14
5.3	Bottom-two and Judges' Save Mechanism	15
5.4	Synthesis of Findings from Rules	15
6	Task D-1: Impact Analysis: Pro Dancers & Celebrity Attributes	15
6.1	Outcome definitions	15
6.2	Modeling results: separating "who judges reward" vs. "who fans support"	16
7	Task D-2: New Rule Design and Stress Testing	17
7.1	Task D-2: New Rule Design and Stress Testing	17
7.1.1	Design objective: professionalism-first, watchability as a constrained secondary goal	17
7.1.2	Rule A (incumbent baseline): combined score determines elimination	17
7.1.3	Rule B (proposed): judges-only gating → fan-aware decision within the risk set	17
7.1.4	Stress testing via replay	18
8	Conclusion	19
9	Memo to Management	19
A	Appendix: Data, Algorithms, and Reproducibility	20
B	Complete Files for Problem B	20

1 Introduction

Dancing With The Stars (DWTS) combines judges’ scores with audience voting to eliminate contestants weekly, yet the audience vote totals remain confidential.[3] This creates a recurring tension between “technical merit” and “popular support,” amplified by format changes across seasons (percent-based versus rank-based aggregation, and the later bottom-two judges’ save). The dataset [1] spans 34 seasons with contestant attributes, weekly judges’ scores, and realized eliminations, but it does not reveal audience votes—turning the central signal of fan preference into a latent variable.

Our key methodological idea is to treat DWTS as a replayable inverse problem. We first construct a canonical season–week–contestant panel with an explicit active set (excluding post-exit zero-score records and flagging special episodes). We then infer weekly fan vote *shares* by solving a constrained optimization problem: elimination-consistency constraints encode the show’s rule mechanics, slack variables absorb exceptional weeks, and a temporal smoothness prior selects a stable solution among many feasible explanations. With inferred fan support in hand, we implement a rule replay engine that simulates eliminations week-by-week under alternative aggregation schemes (rank-based vs. percent-based, with optional judges’ save), enabling direct counterfactual comparisons. The inference module outputs not only point estimates $\hat{p}_{i,t}$ but also uncertainty summaries (e.g., feasible ranges and sensitivity margins). These outputs become direct inputs to the replay engine, closing the loop from latent-vote inference to counterfactual elimination paths and to quantitative evidence for rule comparison and new-system validation.

We evaluate models and voting rules using replay-based metrics that align with the show’s decisions. For vote inference, we report elimination-consistency rates, feasible-range widths (as an identifiability proxy), and sensitivity to regularization strength. For rule comparisons and new-system design, we quantify outcome divergence (who would be eliminated when), shifts in finalist composition, and robustness under stress tests that vary vote weights and the assumed rule-switch season.

To ensure interpretability, we benchmark against simple baselines (e.g., judges-only elimination and naive fan-share models) and require that any added complexity yields measurable improvements in replay fidelity and stability. Building on inferred votes, we fit mixed-effects models to separately explain judges’ scoring and fan preference, isolating professional-dancer effects and celebrity attributes, and we use these insights to design a parameterized voting system that balances fairness, suspense, and robustness.

The remainder of the paper is organized as follows: Section 2 formalizes tasks, assumptions, and notation; Section 3 describes data processing and feature construction; Sections 4–7 present vote inference, rule replay, mechanism analysis, and the proposed voting system with robustness tests; Section 8 concludes; and Section 9 provides a memo to management.

2 Problem Decomposition and Analysis

This section formalizes DWTS as a replayable inverse problem and specifies the modeling objects that close our pipeline. We define the weekly active set, the observable judges’ score signal, and the realized elimination outcomes, then introduce the latent fan vote share $p_{i,t}$ as the primary unknown. Our inference module outputs point estimates $\hat{p}_{i,t}$ together with uncertainty summaries, which feed a unified rule replay engine; this engine generates counterfactual elimination paths used for rule comparison, case-based attribution, and the validation of a proposed new voting system.

2.1 Problem Decomposition

We decompose the prompt into four linked tasks with shared inputs and a common replay-based evaluation layer.

- **Task A: Fan vote inference with uncertainty.** **Input:** weekly judges’ scores and the realized eliminated

set; **Output:** weekly fan vote shares $p_{i,t}$ (and uncertainty bands) over the active set; **Validation:** elimination-consistency under the assumed rule, plus identifiability diagnostics (feasible-range widths and margin-based sensitivity).

- **Task B: Voting-rule comparison via replay.** **Input:** judges' scores and inferred $p_{i,t}$; **Output:** counterfactual week-by-week eliminations under percent-based vs. rank-based aggregation (and variants); **Validation:** season-level and week-level outcome divergence measures and "fan bias" indicators.
- **Task C: Controversial contestants and judges' save attribution.** **Input:** inferred $p_{i,t}$ and replay engine; **Output:** case-study narratives supported by counterfactual trajectories with/without bottom-two judges' save; **Validation:** whether the mechanism explains observed controversies and how sensitive conclusions are to vote uncertainty.
- **Task D: Mechanism analysis and new system design.** **Input:** panel features (celebrity attributes, professional dancer identifiers) and inferred votes; **Output:** mixed-effects estimates for drivers of judges' scoring versus fan preference, and a parameterized voting rule; **Validation:** historical replay, robustness checks, and sensitivity analysis over hyperparameters and rule-switch assumptions.

2.2 Key Assumptions

We adopt a small, testable assumption budget to make the inverse problem well-posed while keeping conclusions falsifiable.

1. **Share normalization.** Fan votes are modeled as within-week shares over the active set.
2. **Rule-governed elimination.** Eliminations follow the documented aggregation rule (percent-based or rank-based), optionally augmented by bottom-two judges' save where applicable.
3. **Active-set definition and special episodes.** Post-exit "zeros" represent inactivity and are excluded from the active set; special episodes are explicitly flagged and may be accommodated via slack rather than forced exact compliance. When a counterfactual replay keeps an observed-exit contestant active, the required post-exit inputs are filled using a conservative, minimal-information imputation; this is used only to extend counterfactual trajectories and is not intended as a performance forecast.
4. **Judges as observed signal.** Judges' scores provide an observed performance signal taken as exogenous to the voting mechanism.
5. **Temporal smoothness.** Fan preference evolves smoothly week-to-week, motivating temporal regularization.

The season at which the show returns to rank-based aggregation is treated as an explicit assumption and is stress-tested using adjacent cutoffs.

2.3 Key Notations

Table 1 summarizes indices, sets, observed quantities, latent fan vote shares, and rule-dependent combined scores used throughout the inference and replay pipeline.

Symbol	Definition
s	Season index, $s = 1, \dots, 34$
t	Week index within season s
i	Contestant (celebrity–pro pair) index
$A_{s,t}$	Active contestants in season s , week t
$E_{s,t}$	Eliminated set at end of week t ; $m_{s,t} = E_{s,t} $
$j_{i,t,k}$	Score given by judge k to contestant i in week t
$J_{i,t}$	Total judges' score: $J_{i,t} = \sum_k j_{i,t,k}$ (skip missing)
$q_{i,t}^J$	Judges' share: $q_{i,t}^J = J_{i,t} / \sum_{r \in A_{s,t}} J_{r,t}$
$p_{i,t}$	Fan vote share (latent), $\sum_{i \in A_{s,t}} p_{i,t} = 1$
α	Weight on judges under percent rule (default $\alpha = 0.5$; stress-tested)
$S_{i,t}^{(P)}$	Percent combined score: $S_{i,t}^{(P)} = \alpha q_{i,t}^J + (1 - \alpha)p_{i,t}$
$R_{i,t}^J, R_{i,t}^V$	Judges rank and fan rank (descending)
$R_{i,t}^{(R)}$	Rank combined score: $R_{i,t}^{(R)} = R_{i,t}^J + R_{i,t}^V$
ξ_t	Slack for elimination-consistency; penalized in vote inference
λ	Penalty weight on slacks in vote inference objective

Table 1: Core notations used throughout the paper.

2.4 Tasks and Evaluation KPIs

Our workflow consists of four tightly coupled tasks. **Task A** reconstructs latent weekly fan vote shares $p_{i,t}$ for each active contestant by solving a season-level constrained optimization problem that enforces elimination consistency (with slack) under the documented mechanics[4, 5]. **Tasks B/C** use these inferred votes to replay each season under alternative aggregation rules (rank-based vs. percent-based, optionally with a bottom-two judges' save), producing counterfactual eliminations and placements. **Task D1** fits mixed-effects models to contrast the drivers of judges' evaluations versus audience support (celebrity attributes and professional partners). **Task D2** proposes an implementable voting rule and selects parameters via replay-based stress tests. To evaluate models and rule variants consistently, we use the following KPIs and slices.

KPI1: Elimination Set Agreement (Jaccard Similarity). On each season–week (s, t) , we compute the replayed eliminated set $\hat{E}_{s,t}$ under a specified rule and compare it to the observed eliminated-at-end-of-week set $E_{s,t}$ parsed from results. We measure set-level agreement by the Jaccard similarity

$$J_{s,t} = \frac{|E_{s,t} \cap \hat{E}_{s,t}|}{|E_{s,t} \cup \hat{E}_{s,t}|}.$$

KPI1 is reported as the week-level mean of $J_{s,t}$ over all counted season–week instances. Weeks with no elimination yield $E_{s,t} = \emptyset$ (and possibly $\hat{E}_{s,t} = \emptyset$); we adopt the convention $J_{s,t} = 1$ when both sets are empty and include such weeks in the mean.

KPI3: Rule Divergence Rate (Rank vs. Percent). Holding the same vote input (e.g., BL-0 proxy or inferred votes), KPI3 measures how often two aggregation schemes disagree on eliminations:

$$\text{KPI3} = \frac{\sum_{(s,t) \in \Omega} \mathbf{1}\{\hat{E}_{s,t}^{\text{rank}} \neq \hat{E}_{s,t}^{\text{percent}}\}}{|\Omega|}, \quad (1)$$

where Ω denotes eligible (s, t) . KPI3 is reported descriptively as “impact magnitude,” not as an optimization objective.

KPI2: Vote Identifiability (interval tightness). Because votes are unobserved, vote inference is not always identifiable.[6, 7] For each (s, t, i) , we compute a feasible interval $[p_{i,t}^{\min}, p_{i,t}^{\max}]$ over fan vote shares consistent with elimination constraints (and with a near-optimal shell around the Task A objective). We define an identifiability score as

$$ID_{i,t} = \frac{1}{p_{i,t}^{\max} - p_{i,t}^{\min} + \varepsilon}, \quad (2)$$

and summarize by quantiles across contestants, weeks, and seasons. KPI2 flags weeks and contestants where multiple vote allocations explain the observed elimination equally well.

Reproducible baseline (BL-0). Before introducing inferred votes, we report KPI1/KPI3 under BL-0 as an audit baseline. In BL-0, the rank scheme reduces to judges’ ranks only (uniform fan ranks add a constant), and the percent scheme uses judges’ percent plus a uniform fan share $1/n_{\text{active}}$. BL-0 validates denominator handling and replay mechanics, but it is not used to claim audience preference estimates.

3 Data Processing and Analysis

3.1 Data Cleaning Principles and Validation

3.1.1 Cleaning Principles

Our preprocessing is *model-driven*: we only perform transformations required to (i) construct a reproducible season–week–contestant panel, (ii) define eligibility/denominator rules for KPI computation, and (iii) enable counterfactual replay under alternative voting schemes. We preserve the dataset’s official encodings and control their impact through an explicit *active-set* definition rather than imputing within-week missing records. A conservative, minimal-information *imputation* is used *only* when a counterfactual replay keeps an observed-exit contestant active beyond the observed data horizon, solely to close the replay loop (*not* to predict future performance) [3].

Principle A — Respect official encodings (0 scores, N/A, irregular season length). The official problem statement clarifies that (i) weeks with no elimination or multiple eliminations exist, (ii) N/A can mean either “no 4th judge” or “weeks beyond a season’s actual run”, and (iii) a score of 0 is recorded for celebrities after elimination (post-exit padding). We therefore treat these as *structural encodings* rather than noise [3].

Principle B — Eligibility controls denominators (active-set construction). A contestant is considered active in (s, t) if at least one judge score exists for that week and the week does not exceed the contestant’s exit week (eliminated week parsed from `results`, or an inferred withdraw week). Post-elimination zeros are retained for auditability but excluded from denominators via `active=false`.

Principle C — Within-week aggregation only; no judge identity tracking. Because judge indices are not persistent identities across weeks/seasons, we only use within-week aggregates (weekly total, rank, and percent) and avoid modeling judge-specific time series at the preprocessing stage [3].

Principle D — Special episodes are handled explicitly. Weeks with zero eliminations are represented by the empty eliminated set and included in KPI computation via an explicit empty-set convention for Jaccard similarity; multi-elimination weeks are evaluated by set-based agreement on the bottom- k eliminated set.

3.1.2 Cleaning Principles

Our preprocessing is *model-driven*: we only perform transformations required to build a reproducible season–week–contestant panel and to support eligibility-aware KPI computation and rule replays. We adopt the following principles (write rules, not results):

1. **Respect official encodings (0 vs. N/A).** Post-elimination zeros are treated as structural padding rather than new performances; their influence is removed via the `active` definition. N/A indicates the absence of a judge in that week (e.g., only three judges) or out-of-range weeks; we aggregate with `skipna` and do not fill missing judges.
2. **Active-set driven accounting (denominator control).** All within-week percentages, replay denominators, and KPI denominators are computed on the set `active=1`. Records after a contestant exits are excluded from denominators even if they exist in the raw encoding.
3. **No aggressive within-week imputation.** During cleaning we do not impute missing judge scores and do not “invent” performance values; we only apply deterministic reshaping and aggregation. (Any “imputation” mentioned elsewhere is solely for closing counterfactual replays on unobserved post-exit segments, not for forecasting.)
4. **Special weeks are handled explicitly.** Weeks with zero eliminations and weeks with multiple eliminations are treated as first-class cases in KPI computation and replays; we do not force them into a single-elimination template.
5. **Deterministic and reproducible.** The transformation from raw wide-format inputs to the canonical replay-ready panel is deterministic; the same raw input yields the same canonical output.

3.1.3 Data Cleaning Outcomes and Validation

Outcomes (artifacts and interfaces). The output of preprocessing is a *canonical replay-ready weekly panel* (`outputs/data_cleaned/canonical_replay_ready.csv`) that contains eligibility/exit labels (`active`, `exit_type`, `exit_week`, `true_elim_flag`) and judge-based within-week features (`total_judge_score`, `judge_rank`, `judge_percent`). This panel is the single input interface used for all downstream KPI recomputation, vote-share inference, and replay simulation.

We also export two compatible variants for engineering convenience:

`outputs/data_cleaned/clean_long_data_replay_ready.csv` (replay-ready alias with aligned field names) and `outputs/data_cleaned/clean_long_data_new1.csv` (clean long panel with additional friendly fields such as `J_total`, `J_pct`, and `elim_week`).

In addition, we generate report-ready artifacts for baseline auditing:

- (i) a $\LaTeX$ table `outputs/tab/tab_baseline_consistency.tex`
- (ii) an optional figure `outputs/fig/fig_fliprate_by_season.pdf`.

Validation (what we check, and why it is sufficient for P0). We validate preprocessing through three audit layers:

- **(V1) Encoding-consistency checks.** We follow the official semantics for N/A and post-elimination zeros, preserving these encodings and excluding them from denominators via `active` [3].
- **(V2) Denominator/eligibility invariants.** `judge_percent` is computed only among active contestants within each (*season, week*); when total score is positive, it sums to one within the active set. The explicit `data_anomaly_zero_score` flag prevents accidental inclusion of padded zeros.
- **(V3) End-to-end reproducibility via BL-0 sanity check.** On top of the canonical panel, we implement the BL-0 baseline (rank: judge-rank only; percent: judge-percent plus uniform $1/n_{\text{active}}$) and reproduce a consistent set of KPI1/KPI3 summaries across era slices. This serves as a pipeline sanity check rather than a final model claim.

3.2 Panel Construction

Panel construction converts the wide-format score table into a long-format dataset where each record corresponds to a unique (*season, week, celebrity*) unit. We reshape `weekX_judgeY_score` into such a long panel and compute weekly aggregates (`total_judge_score`) and within-week relative measures (`judge_rank`, `judge_percent`) among the active set. We then attach deterministic exit/eligibility labels (`exit_type`, `exit_week`, `true_elim_flag`, `active`) derived from `results` and score availability. The resulting canonical panel (`outputs/data_cleaned/canonical_replay_ready.csv`) is the shared input for all subsequent modules, including vote-share inference and counterfactual replay under alternative voting schemes.

3.3 Empirical Findings from EDA (only what feeds models)

3.3.1 Data Visualization

We summarize baseline consistency and rule divergence using:

- **(L1 table)** `outputs/tab/tab_baseline_consistency.tex`: BL-0 baseline set agreement (KPI1: Jaccard similarity) under rank and percent schemes, and rule divergence (KPI3/FlipRate), with slices: Overall / Rank-era / Percent-era.
- **(L2 figure, optional)** `outputs/fig/fig_fliprate_by_season.pdf`: season-level visualization of where the two schemes disagree.

3.3.2 Key Empirical Findings

We report only findings that directly inform the model design and evaluation:

Finding 1 — A reproducible BL-0 baseline exists and is non-trivial. Using only judge-based information and a uniform fan proxy, BL-0 yields measurable elimination-consistency KPI1 under both rank and percent aggregation, and a non-zero flip rate between the two schemes. These quantities are used strictly as a sanity check for the pipeline and as a reference point for later vote-inference models; minor changes may occur if the canonical panel definition is updated, but the purpose is to validate consistent eligibility labeling and replay mechanics. All substantive conclusions in later sections are based on the inferred vote shares and the counterfactual replay framework, not on BL-0.

Finding 2 — Rule comparisons must be stress-tested around the season-28 cutoff. We adopt the era split for rule-comparison slicing (Rank-era vs. Percent-era), but treat the Season-28 switch as an explicit assumption and stress-test adjacent cutoffs (27/28/29).

Finding 3 — Non-standard weeks materially affect set-based consistency metrics. Because the dataset includes weeks with no elimination or multiple eliminations, KPI1 must handle these episodes explicitly (e.g., empty-set convention for no-elimination weeks and set-based comparison for multi-elimination weeks). Such episodes can introduce non-uniform weighting under aggregation, so we interpret KPI1 primarily as a replay-pipeline consistency audit metric and emphasize relative comparisons under the same convention.

Implementation note (KPI1: Jaccard similarity, week-level mean). For each season-week (s, t), let $E_{s,t}$ denote the observed eliminated set (possibly empty) and $\hat{E}_{s,t}$ the replayed eliminated set under a

specified rule. We measure set-level agreement by the Jaccard similarity

$$J_{s,t} = \frac{|E_{s,t} \cap \hat{E}_{s,t}|}{|E_{s,t} \cup \hat{E}_{s,t}|}.$$

KPI1 is reported as the week-level mean of $J_{s,t}$ over all counted season-week instances. Weeks with no elimination yield $E_{s,t} = \emptyset$ (and possibly $\hat{E}_{s,t} = \emptyset$), for which the ratio becomes 0/0; we adopt the convention $J_{s,t} = 1$ when both sets are empty (full agreement on “no elimination”) and include such weeks in the mean. Due to aggregation details, some no-elimination weeks may be counted more than once, introducing an additional weighting effect that can shift the absolute level of KPI1. We therefore interpret KPI1 primarily as a replay-pipeline consistency audit metric, and emphasize relative comparisons across rules and stress tests under the same convention.

3.4 Feature Engineering

We keep feature engineering minimal and aligned with downstream modeling needs:

- **Performance signals (week-level, within active set).** `total_judge_score`, `judge_rank`, and `judge_percent` are computed per (*season*, *week*) among active contestants and serve as the observed performance component when replaying elimination rules or inferring latent vote shares.
- **Explanatory covariates (contestant-level, for heterogeneity analysis).** When available, we use `ballroom_partner` (pro dancer identity), `celebrity_age_during_season`, `celebrity_industry`, and region-related fields for slicing and mixed-effects heterogeneity analyses.

3.5 Synthesis of Findings

In summary, our preprocessing produces a canonical season-week-contestant panel that (i) respects the official encodings of N/A and post-elimination zeros, (ii) enforces denominator correctness through an explicit active-set definition, and (iii) enables reproducible baseline replay consistent with our KPI definitions. Key data caveats—non-standard elimination weeks, varying judge availability, and the uncertain Season-28 era cutoff—are surfaced explicitly for robustness stress tests rather than hidden by aggressive cleaning [3].

4 Task A: Fan Vote Inference Model

We propose a deployable, data-driven fan vote inference model. The key design principle is to **treat audience votes as a latent signal and infer vote shares from observed judges’ scores and elimination outcomes under explicit rule constraints**.

4.1 Overview and Modeling Philosophy

Fan votes are a latent signal and must be inferred indirectly. In DWTS, eliminations depend on a combination of judges’ scores and audience votes, yet only judges’ scores and realized eliminations are observed. We therefore model weekly audience support as a latent vote-share vector over the active set. Because many vote configurations can be consistent with the same elimination outcome, the inference problem is generally underdetermined; our approach resolves this by (i) enforcing elimination-consistency constraints implied by the show’s rule mechanics and (ii) selecting a stable solution via temporal smoothness regularization.[6, 7] Special episodes (e.g., no-elimination or irregular-format weeks) are handled explicitly through slack variables rather than forcing exact compliance, and these slacks become a diagnostic for identifiability and model mismatch.

4.2 Main Inference Model

Decision variables. For each season s , week t , and active contestant $i \in A_{s,t}$, let $p_{i,t} \geq 0$ denote the unobserved fan vote share, with $\sum_{i \in A_{s,t}} p_{i,t} = 1$. Let $J_{i,t}$ be the observed total judges' score (sum across judges with missing values skipped) and define the judges' share $q_{i,t}^J = J_{i,t} / \sum_{r \in A_{s,t}} J_{r,t}$.

Percent-scheme combined score (primary inference target). Under the percent-based aggregation, we define the combined score

$$S_{i,t}^{(P)} = \alpha q_{i,t}^J + (1 - \alpha) p_{i,t}, \quad (3)$$

where $\alpha \in [0, 1]$ controls the relative weight of judges versus fans (default $\alpha = 0.5$; later stress-tested).

Elimination-consistency constraints with slack. Let $E_{s,t} \subseteq A_{s,t}$ denote the observed eliminated set at the end of week t with $m_{s,t} = |E_{s,t}|$. We introduce a nonnegative slack $\xi_t \geq 0$ and impose

$$S_{e,t}^{(P)} \leq S_{i,t}^{(P)} + \xi_t, \quad \forall e \in E_{s,t}, \forall i \in A_{s,t} \setminus E_{s,t}. \quad (4)$$

When the assumed rule and data are perfectly compatible, the minimum feasible ξ_t is zero; otherwise $\xi_t > 0$ quantifies the smallest violation needed to reconcile the observed outcome.[8]

Objective with judge-anchor prior. To avoid degenerate “nearly-uniform” vote reconstructions, we add a judge-anchor prior that pulls fan vote shares toward judges' score shares in the absence of strong contradictory evidence. For each season s , we solve:

$$\begin{aligned} \min_{\{p_{i,t}\}, \{\xi_t\}} \quad & \underbrace{\sum_{t=2}^{T_s} \sum_{i \in A_{s,t} \cap A_{s,t-1}} (p_{i,t} - p_{i,t-1})^2}_{\text{Temporal smoothness}} + \underbrace{\lambda_1 \sum_{t=1}^{T_s} \sum_{i \in A_{s,t}} (p_{i,t} - q_{i,t}^J)^2}_{\text{Judge anchor prior}} + \underbrace{\lambda_2 \sum_{t=1}^{T_s} \xi_t^2}_{\text{Violation penalty}} \quad (5) \\ \text{s.t.} \quad & \sum_{i \in A_{s,t}} p_{i,t} = 1, \quad p_{i,t} \geq 0, \quad \xi_t \geq 0, \quad \forall t, \\ & S_{e,t}^{(P)} \leq S_{k,t}^{(P)} + \xi_t, \quad \forall t, \forall e \in E_{s,t}, \forall k \in A_{s,t} \setminus E_{s,t}. \end{aligned}$$

Remark (objective form). Equation (5) is a convex quadratic program (QP) solved independently for each season. The smoothness term is computed only for contestants active in consecutive weeks ($A_{s,t} \cap A_{s,t-1}$), while the anchor and slack penalties apply to all active contestants and weeks. Hyperparameters (λ_1, λ_2) are selected by replay-based diagnostics (Section 4.4).[4, 5]

Algorithm 1 Season-wise fan vote inference via constrained QP

Require: Weekly panel for season s : active sets $A_{s,t}$, judges' totals $J_{i,t}$, eliminated sets $E_{s,t}$; hyperparameters $\alpha, \lambda_1, \lambda_2$.

Ensure: Point estimates $\hat{p}_{i,t}$ and slack diagnostics $\hat{\xi}_t$ for all eligible weeks.

1: Compute judges' share for each week t :

$$q_{i,t}^J = \frac{J_{i,t}}{\sum_{r \in A_{s,t}} J_{r,t}}$$

2: Define decision variables $\{p_{i,t}\}_{i \in A_{s,t}}$ with constraints:

$$p_{i,t} \geq 0, \quad \sum_{i \in A_{s,t}} p_{i,t} = 1$$

3: For each week t , define percent-based composite score $S_{i,t}^{(P)}$ and slack variable ξ_t :

$$S_{i,t}^{(P)} = \alpha q_{i,t}^J + (1 - \alpha) p_{i,t}, \quad \xi_t \geq 0$$

4: Add elimination-consistency constraints:

$$S_{e,t}^{(P)} \leq S_{i,t}^{(P)} + \xi_t \quad \forall e \in E_{s,t}, i \in A_{s,t} \setminus E_{s,t}$$

5: Solve the quadratic program (QP) to minimize the objective function:

$$\mathcal{L} = \sum_{t=2}^{T_s} \sum_{i \in A_{s,t} \cap A_{s,t-1}} (p_{i,t} - p_{i,t-1})^2 + \lambda_1 \sum_{t=1}^{T_s} \sum_{i \in A_{s,t}} (p_{i,t} - q_{i,t}^J)^2 + \lambda_2 \sum_{t=1}^{T_s} \xi_t^2$$

6: Output $\hat{p}_{i,t}$ (inferred vote shares) and $\hat{\xi}_t$ (slack diagnostics); flag weeks with large $\hat{\xi}_t$ as potential special-episode mismatches.

4.3 Uncertainty Quantification and Sensitivity Analysis

Uncertainty is intrinsic because the inverse problem is not fully identifiable. We report three complementary uncertainty summaries: (i) **Feasible intervals:** for each (i, t) , compute the minimum and maximum $p_{i,t}$ attainable under the same constraints (yielding an interval width as an identifiability proxy, KPI2); (ii) **Elimination margin:** quantify how much perturbation to $\hat{p}_{i,t}$ is required to flip the predicted eliminated set under the replay engine, providing a week-level stability score; and (iii) **Slack diagnostics:** ξ_t indicates weeks where observed outcomes cannot be matched under the assumed mechanics without violation; these weeks are flagged for special-episode handling and for sensitivity analysis over the rule-switch season and $\alpha, \lambda_1, \lambda_2$.

4.3.1 KPI2: Vote Identifiability (interval tightness)

KPI2: Vote Identifiability (interval tightness). The KPI registry defines vote identifiability by the tightness of a feasible interval for each (s, t, i) under an explicit elimination mechanism. In Task A, we instantiate KPI2 under the **percent-based rule** with the fan vote **share** variable $p_{i,t}$, and set $\varepsilon = 10^{-6}$ to avoid singularities. After solving the season-wise inference QP and obtaining $(\hat{p}, \hat{\xi})$ with optimal objective value OPT_s , we compute a **near-optimal feasible interval** for each active contestant-week pair by solving two auxiliary profile problems:

$$p_{i,t}^{\min} = \min p_{i,t}, \quad p_{i,t}^{\max} = \max p_{i,t},$$

subject to the original probability and elimination-consistency constraints, plus a tolerance shell that keeps solutions comparable to the fitted trajectory:

$$\mathcal{L}(p, \xi) \leq (1 + \delta_{\text{obj}}) \text{OPT}_s, \quad \xi_\tau \leq \hat{\xi}_\tau + \delta_\xi \quad (\forall \tau),$$

where $\mathcal{L}(p, \xi)$ is the Task-A objective (smoothness + judge-anchor + slack penalty). The interval width $w_{i,t} = p_{i,t}^{\max} - p_{i,t}^{\min}$ is mapped to an identifiability score

$$\text{ID}_{i,t} = \frac{1}{w_{i,t} + \varepsilon}.$$

We summarize $\text{ID}_{i,t}$ by median/quantiles across contestants and weeks and aggregate by season and by early/mid/late phases; in implementation we compute KPI2 on selected slices (e.g., late-phase weeks or controversial cases) to control computational cost.

Profile optimization formulation. For a fixed season s , let T_s be the number of weeks. Define decision variables

$$\mathbf{p} = \{p_{j,\tau} : j \in A_{s,\tau}, \tau = 1, \dots, T_s\}, \quad \boldsymbol{\xi} = \{\xi_\tau : \tau = 1, \dots, T_s\}.$$

For each week τ , the percent-rule combined score is

$$S_{j,\tau}^{(P)} = \alpha q_{j,\tau}^J + (1 - \alpha) p_{j,\tau}.$$

Base constraints. We impose simplex constraints for each week:[8–10]

$$p_{j,\tau} \geq 0, \quad \forall j \in A_{s,\tau}, \forall \tau, \quad (6)$$

$$\sum_{j \in A_{s,\tau}} p_{j,\tau} = 1, \quad \forall \tau. \quad (7)$$

Slacks are nonnegative:

$$\xi_\tau \geq 0, \quad \forall \tau. \quad (8)$$

Elimination-consistency constraints (same as Task A) are

$$S_{e,\tau}^{(P)} \leq S_{k,\tau}^{(P)} + \xi_\tau, \quad \forall \tau, \forall e \in E_{s,\tau}, \forall k \in A_{s,\tau} \setminus E_{s,\tau}. \quad (9)$$

Near-optimal shell (to prevent uninformative, overly wide intervals). Let the Task-A objective be

$$\mathcal{L}(\mathbf{p}, \boldsymbol{\xi}) = \sum_{\tau=2}^{T_s} \sum_{j \in A_{s,\tau} \cap A_{s,\tau-1}} (p_{j,\tau} - p_{j,\tau-1})^2 + \lambda_1 \sum_{\tau=1}^{T_s} \sum_{j \in A_{s,\tau}} (p_{j,\tau} - q_{j,\tau}^J)^2 + \lambda_2 \sum_{\tau=1}^{T_s} \xi_\tau^2. \quad (10)$$

After solving the main inference QP, we obtain $(\hat{\mathbf{p}}, \hat{\boldsymbol{\xi}})$ and $\text{OPT}_s = \mathcal{L}(\hat{\mathbf{p}}, \hat{\boldsymbol{\xi}})$. To compute intervals around the fitted explanation, we add:

$$\mathcal{L}(\mathbf{p}, \boldsymbol{\xi}) \leq (1 + \delta_{\text{obj}}) \text{OPT}_s, \quad (11)$$

$$\xi_\tau \leq \hat{\xi}_\tau + \delta_\xi, \quad \forall \tau. \quad (12)$$

In implementation, (11) can be written as a convex SOC constraint by noting that $\mathcal{L}$ is a sum of squares:

$$\left\| \begin{bmatrix} \{p_{j,\tau} - p_{j,\tau-1}\}_{\tau=2..T_s, j \in A_{s,\tau} \cap A_{s,\tau-1}} \\ \{\sqrt{\lambda_1} (p_{j,\tau} - q_{j,\tau}^J)\}_{\tau=1..T_s, j \in A_{s,\tau}} \\ \{\sqrt{\lambda_2} \xi_\tau\}_{\tau=1..T_s} \end{bmatrix} \right\|_2 \leq \sqrt{(1 + \delta_{\text{obj}}) \text{OPT}_s}.$$

This SOC form is directly supported by standard convex solvers (e.g., via CVX frameworks).[11]

Profile-min and profile-max. For each active pair (i, t) with $i \in A_{s,t}$, define:

$$p_{i,t}^{\min} = \min_{\mathbf{p}, \xi} p_{i,t} \quad (13)$$

s.t. (6) – (9), (11) – (12),

and

$$p_{i,t}^{\max} = \max_{\mathbf{p}, \xi} p_{i,t} \quad (14)$$

s.t. (6) – (9), (11) – (12).

We then set the interval width $w_{i,t} = p_{i,t}^{\max} - p_{i,t}^{\min}$ and identifiability score $ID_{i,t} = 1/(w_{i,t} + \varepsilon)$.

Algorithm 2 KPI2 computation via profile optimization (per season)

Require: Season s weekly panel: active sets $A_{s,t}$, eliminated sets $E_{s,t}$, judges' shares $q_{i,t}^J$; main-model hyperparameters $\alpha, \lambda_1, \lambda_2$; KPI2 params $\varepsilon, \delta_{\text{obj}}, \delta_{\xi}$; a slice selector $\mathcal{S}$ (set of (i, t) pairs to evaluate).

Ensure: Interval bounds $\{p_{i,t}^{\min}, p_{i,t}^{\max}\}$ and identifiability scores $\{ID_{i,t}\}$ for $(i, t) \in \mathcal{S}$.

- 1: **Solve main inference QP** for season s to obtain $(\hat{\mathbf{p}}, \hat{\xi})$ and $\text{OPT}_s = \mathcal{L}(\hat{\mathbf{p}}, \hat{\xi})$.
 - 2: **Define common constraints:** simplex constraints (7)–(9) and slack nonnegativity (8).
 - 3: **Add KPI2 shells:** objective shell (11) (or its SOC form) and slack cap (12).
 - 4: Choose evaluation slice $\mathcal{S}$ (e.g., late-phase weeks and/or controversial cases) to control computation.
 - 5: **for** each $(i, t) \in \mathcal{S}$ **do**
 - 6: Solve profile-min optimization (13) to get $p_{i,t}^{\min}$.
 - 7: Solve profile-max optimization (14) to get $p_{i,t}^{\max}$.
 - 8: Compute width $w_{i,t} = p_{i,t}^{\max} - p_{i,t}^{\min}$ and identifiability score $ID_{i,t} = 1/(w_{i,t} + \varepsilon)$.
 - 9: Sanity check: verify $p_{i,t}^{\min} \leq \hat{p}_{i,t} \leq p_{i,t}^{\max}$; if violated, align constraints with the main QP.
 - 10: **end for**
 - 11: Aggregate $ID_{i,t}$ by week/season/phase using medians and quantiles as defined in the KPI registry.
-

4.4 Hyperparameters, tuning, and implementation details

Hyperparameters and defaults. Task A uses α (judge weight in the percent rule), λ_1 (strength of the judge-anchor prior), λ_2 (penalty on constraint violations via ξ_t), and the KPI2 parameters $\varepsilon, \delta_{\text{obj}}, \delta_{\xi}$. Unless otherwise stated, we set $\alpha = 0.5$ as the nominal DWTS percent weighting and treat α as a sensitivity knob in later stress tests.

We tune (λ_1, λ_2) by replay-based diagnostics. Increasing λ_2 enforces stricter elimination-consistency (smaller ξ_t) at the cost of reduced feasibility for irregular-format weeks; decreasing λ_2 allows the model to absorb format mismatches but may underfit eliminations. Increasing λ_1 pulls inferred votes toward judges' score shares and helps avoid near-uniform solutions, while excessively large λ_1 may suppress genuine judge–fan disagreements that are evidenced by the observed eliminations. Practically, we tune (λ_1, λ_2) on a season-by-season basis by balancing (i) the frequency and magnitude of nonzero slacks $\hat{\xi}_t$, (ii) the stability of inferred top- k vote shares across adjacent weeks, and (iii) downstream replay fidelity in Task B.

Special episodes and multi-elimination weeks. Our formulation supports (a) multi-elimination weeks by allowing $|E_{s,t}| > 1$ and enforcing the same pairwise inequalities against all non-eliminated contestants, and (b) non-elimination or irregular weeks by allowing $E_{s,t} = \emptyset$ (no elimination constraints) while still solving the smoothness objective; such weeks can also be explicitly flagged and assessed via slack and sensitivity tests. If a season contains weeks whose outcomes cannot be reconciled with the assumed mechanics, the model will surface this via persistently large $\hat{\xi}_t$; these weeks are treated as format exceptions rather than forcing ad hoc re-labeling.

Solver and numerical stability. The main inference problem is a convex QP and can be solved season-wise using standard QP solvers. The KPI2 shell can be implemented either directly as a quadratic constraint or in SOC form; the latter is solver-friendly in CVX frameworks. To avoid numerical issues, we recommend (i) normalizing judges' scores into shares q^J (already done), (ii) using $\varepsilon = 10^{-6}$ for identifiability scoring, and (iii) keeping δ_{obj} small (e.g., 1–5%) so profile intervals remain comparable to the fitted explanation.

5 Task B & C: Rule Reply, Volting System, and Counterfactuals

Following the official rule definitions, we perform season–week counterfactual replays under (i) rank-based aggregation and (ii) percent-based aggregation. We initialize each replay with the observed eligibility labels, but update the active set week-by-week according to the counterfactual eliminations under the chosen rule. When a counterfactual path keeps an observed-exit contestant active beyond the observed data horizon, we apply a conservative, minimal-information imputation solely to complete the replay (not to forecast future performance). As a reproducible sanity-check baseline (BL-0), we use judge-only information with a uniform fan proxy and report elimination hit-rate (KPI1) and the rule divergence rate (KPI3/FlipRate). This chapter summarizes cross-season differences (Section 5.1), explains divergences via controversial case studies (Section 5.2), evaluates the incremental impact of bottom-two and judges' save (Section 5.3), and synthesizes implications (Section 5.4).

5.1 Rank vs. Percent Across Seasons

Results in this section are first reported under BL-0 as a reproducible sanity check; Task A inferred votes are used in subsequent replays.

5.1.1 Replay protocol and eligibility

Goal. This section quantifies how much the season outcomes depend on the aggregation rule by replaying each season under (i) percent-based aggregation and (ii) rank-based aggregation while holding inputs constant. For every season–week, we restrict attention to the *active set* $A_{s,t}$ and use judges' information (e.g., total judges' score or its normalized share) together with the inferred fan vote shares $\hat{p}_{i,t}$ from Task A. Given an elimination count $m_{s,t}$ (including $m_{s,t} = 0$ for non-elimination weeks and $m_{s,t} > 1$ for multi-elimination weeks), the replay engine deterministically selects the bottom $m_{s,t}$ contestants under the chosen rule and updates the active set for the next week.

Conservative imputation for counterfactual survivors. In counterfactual replays, a rule change may keep a contestant active after the week where they exited in the observed data; in that case, the contestant's subsequent-week inputs are unobserved. To complete the replay without injecting trend assumptions, we use a minimal-information imputation: (i) the next-week judges' total score is filled by the contestant's mean-to-date,

$$\hat{J}_{i,t+1} = \frac{1}{t} \sum_{k=1}^t J_{i,k},$$

and (ii) the next-week fan vote share is carried forward from the most recent inferred value,

$$\hat{p}_{i,t+1} = \hat{p}_{i,t}.$$

This imputation is used only to extend counterfactual trajectories and is not intended to forecast future performance; consequently, any rule-comparison statistics that depend on these imputed segments should be interpreted as conservative estimates of rule impact.

Percent rule. Under percent-based aggregation, the combined score is

$$S_{i,t}^{(P)}(\alpha) = \alpha q_{i,t}^J + (1 - \alpha) \hat{p}_{i,t},$$

and the replay eliminates the $m_{s,t}$ contestants with the smallest $S_{i,t}^{(P)}$.

Rank rule. Under rank-based aggregation, we compute judges' rank $R_{i,t}^J$ from total judges' score $J_{i,t}$ (descending), and vote rank $R_{i,t}^V$ from $\hat{p}_{i,t}$ (descending). The combined rank is

$$R_{i,t}^{(R)} = R_{i,t}^J + R_{i,t}^V,$$

and the replay eliminates the $m_{s,t}$ contestants with the largest $R_{i,t}^{(R)}$ (worst combined ranks).

Tie-handling. To keep comparisons reproducible, ties at the elimination cutoff are resolved deterministically. For percent scores, ties are broken by higher judges' share q^J (survives), and if still tied by a stable identifier order. For rank scores, we use standard competition ranking for tied values and apply the same deterministic tie-breaker when selecting the bottom $m_{s,t}$. All ties are resolved deterministically (details in Appendix X) to ensure replay reproducibility.

5.1.2 Cross-season summary metrics

We compute KPI1/KPI3 over counted (season, week) instances under a unified replay convention. KPI1 (Jaccard set agreement) summarizes week-level agreement between the observed eliminated set and the replayed eliminated set, including no-elimination weeks via an explicit empty-set convention. KPI3 (FlipRate) measures how often the predicted eliminated set differs between percent-based and rank-based replays under the same inputs.

5.1.3 Era cutoff assumption and stress-test plan

Because the exact season boundary of the return to rank-based aggregation may affect interpretation, we treat the era cutoff as an explicit assumption and stress-test adjacent cutoffs (e.g., shifting the boundary by ± 1 season). Conclusions that remain stable under this perturbation are reported as robust, while boundary-sensitive findings are explicitly qualified.

5.2 Controversial Contestants: Case Studies

We select cases from seasons with high FlipRate and large judge-fan discordance, ensuring that each case demonstrates a distinct mechanism.

Purpose. Aggregate FlipRate identifies *where* rules diverge; case studies explain *why* the divergence occurs and how it translates into public controversy. We analyze a small set of representative contestants (e.g. Jerry Rice and Bobby Bones) whose outcomes are commonly discussed in the context of judge-fan disagreement. For each case, we follow a fixed protocol:

- **Observed trajectory:** week-by-week judges' rank, survival, and final placement.
- **Counterfactual replays:** percent vs. rank under the same inferred $\hat{p}_{i,t}$, and (when applicable) with/without a judges' save module.
- **Mechanism attribution:** identify the minimal driver of outcome change (e.g., persistent advantage in vote rank that dominates under rank aggregation, or "rank compression" effects when many contestants cluster in judges' scores).
- **Uncertainty qualification:** if the week-level vote interval is wide (KPI2) or replay margins are small, we report the case as sensitive and avoid over-claiming.

This structure ensures that each narrative claim is backed by a reproducible replay trace rather than anecdotal reasoning.

5.3 Bottom-two and Judges' Save Mechanism

Some later seasons introduce a two-stage mechanism: (i) identify a bottom subset using the vote aggregation rule, then (ii) select the eliminated contestant from that subset via judges' save. To isolate the incremental effect of this change, we implement a modular judges' save layer that can be toggled on top of either percent-based or rank-based aggregation.

Module definition. In a week with a bottom-two format, the replay first forms a bottom-two set $B_{s,t}$ using the chosen aggregation rule. The saved contestant is the one with higher judges' score $J_{i,t}$ (equivalently higher $q_{i,t}^J$), and the other is eliminated. This construction cleanly separates *how the bottom set is formed* from *how the final elimination is chosen*, enabling controlled counterfactual comparisons.

Evaluation questions. We quantify (a) how often the judges' save reverses the elimination implied by votes alone, (b) whether the save disproportionately protects contestants with strong judges' scores but weak fan support, and (c) how the mechanism affects overall rule divergence and late-season stability. These results connect directly to management decisions about perceived fairness versus entertainment value.[2, 12]

5.4 Synthesis of Findings from Rules

We synthesize the rule replay evidence into four actionable takeaways.

(1) Rule choice matters most when judges and fans disagree. Weeks with large judge–fan rank discordance concentrate a disproportionate share of rule flips; in such weeks, percent-based aggregation tends to preserve score magnitude information, while rank-based aggregation amplifies relative ordering.

(2) Divergence is not uniform across the season. Rule sensitivity often increases in late-season weeks when the field is smaller and elimination margins narrow; this motivates reporting uncertainty and replay margins alongside any counterfactual claim.

(3) Judges' save changes the source of authority. A bottom-two + judges' save mechanism shifts the final elimination decision toward judges, reducing the ability of extreme popularity alone to determine outcomes in the bottom set—at the cost of potentially weakening audience agency.

(4) Implication for redesign. These patterns motivate a new aggregation system (Task D2) that preserves interpretability and replay stability while explicitly controlling how much popularity can overwhelm technical performance. The next chapters use the same replay engine to evaluate proposed parameters under robustness and stress tests.

6 Task D-1: Impact Analysis: Pro Dancers & Celebrity Attributes

6.1 Outcome definitions

To disentangle how outcomes are shaped by professional partners versus celebrity attributes, we model judges' evaluations and inferred fan preferences as two distinct but related outcomes. For comparability across seasons and weeks, we work with *shares* rather than raw scores. Let $A_{s,t}$ denote the active set in season s , week t .

Judges' outcome. Define the judges' score share

$$q_{i,t}^J = \frac{J_{i,t}}{\sum_{r \in A_{s,t}} J_{r,t}}, \quad (15)$$

and use a logit transform with a small ϵ to avoid boundary issues:

$$y_{i,t}^J = \log \frac{q_{i,t}^J + \epsilon}{1 - q_{i,t}^J + \epsilon}. \quad (16)$$

Fans' outcome. Let Task A provide the inferred vote share $p_{i,t}$ (denoted $q_{i,t}^V$). We similarly define

$$y_{i,t}^V = \log \frac{p_{i,t} + \epsilon}{1 - p_{i,t} + \epsilon}. \quad (17)$$

We adopt share + logit for cross-season comparability and interpret coefficients on the log-odds scale.

6.2 Modeling results: separating “who judges reward” vs. “who fans support”

We fit two mixed-effects models on the season-week panel to attribute variation to (i) celebrity covariates and (ii) professional dancer partners.^[13]

Model 1: Judge Model.

$$y_{i,t}^J = \beta_0 + \beta_X^\top X_i + \beta_T f(t) + FE_s + u_i + v_{d(i)} + \epsilon_{i,t}, \quad (18)$$

where X_i collects time-invariant celebrity attributes (e.g., age, occupation indicators), $f(t)$ captures week trend, FE_s is a season fixed effect, $u_i \sim \mathcal{N}(0, \sigma_u^2)$ is a celebrity random effect, and $v_{d(i)} \sim \mathcal{N}(0, \sigma_v^2)$ is a professional-dancer random effect. A key hypothesis test is $H_0 : \sigma_v^2 = 0$ (no pro-dancer effect on judges).

Model 2: Vote Model. To model inferred fan support while allowing mediation through judges' performance, we include $y_{i,t}^J$:

$$y_{i,t}^V = \gamma_0 + \gamma_J y_{i,t}^J + \gamma_X^\top X_i + \gamma_T f(t) + FE_s + a_i + b_{d(i)} + \eta_{i,t}, \quad (19)$$

with $a_i \sim \mathcal{N}(0, \sigma_a^2)$ and $b_{d(i)} \sim \mathcal{N}(0, \sigma_b^2)$ capturing celebrity and pro-dancer random effects on fan preferences, respectively. The coefficient γ_J measures the transmission strength from judges to fans.

Comparing judges vs. fans. For each shared attribute k , define the preference gap

$$\Delta_k = \gamma_k - \beta_k. \quad (20)$$

$\Delta_k > 0$ indicates fans prefer the attribute more than judges do; $\Delta_k < 0$ indicates the opposite. We report Δ_k with uncertainty bands (via bootstrap) whenever vote inference uncertainty is non-negligible.

Quantifying professional-dancer impact (magnitude, not just significance). We quantify the fraction of outcome variance attributable to professional dancers using ICC-style ratios:

$$ICC_{pro}^J = \frac{\sigma_v^2}{\sigma_v^2 + \sigma_u^2 + \sigma^2}, \quad ICC_{pro}^V = \frac{\sigma_b^2}{\sigma_b^2 + \sigma_a^2 + \sigma_\eta^2}. \quad (21)$$

We treat BLUP rankings as descriptive summaries of the fitted random effects rather than causal estimates to compare whether the same pros are “good for judges” and “good for fans.”

Uncertainty propagation. Because fan votes are inferred, we propagate uncertainty either by (i) bootstrap re-estimation of votes and refitting the Vote Model, or (ii) weighted regression using the interval width from Task A as an uncertainty proxy; we report confidence intervals accordingly.

7 Task D-2: New Rule Design and Stress Testing

7.1 Task D-2: New Rule Design and Stress Testing

7.1.1 Design objective: professionalism-first, watchability as a constrained secondary goal

Because DWTS is fundamentally a performance competition, we adopt a *professionalism-first* rule design principle: eliminations should be predominantly driven by judges’ performance signals. At the same time, DWTS is a televised product, so the system must preserve watchability by keeping a meaningful audience role and maintaining suspense. Importantly, any additional uncertainty must remain *performance-grounded* (induced by explicit rule structure operating on low performers), rather than arbitrary randomness.

Operational definitions. We define **professionalism** as the extent to which eliminated contestants are concentrated in the lower tail of the weekly judges’ ranking. We define **watchability** as a constrained secondary objective that requires (i) a non-trivial marginal effect of fan votes on survival (audience influence) and (ii) suspense that avoids overly deterministic outcomes, while remaining performance-grounded.

Notation. For each season–week (s, t) , let $q_{i,t}^J$ denote contestant i ’s normalized judges’ score share within the active set, and let $p_{i,t}$ denote the inferred/proxy fan vote share from Task A. For a weight parameter $w_J \in [0, 1]$, define a combined score

$$S_{i,t} = w_J q_{i,t}^J + (1 - w_J) p_{i,t}.$$

7.1.2 Rule A (incumbent baseline): combined score determines elimination

Rule A represents the show’s baseline mechanism, in which judges’ scores and fan votes are combined into a single weekly decision score, and elimination is determined directly by that combined ordering:

1. Compute $S_{i,t}$ for all active contestants.
2. Eliminate the contestant with the lowest $q_{i,t}^J$.

This baseline can preserve audience influence when fan weights are large, but it also permits “popularity-only” survival: a contestant with weak judges’ performance may avoid elimination if fan support is sufficiently strong, which can weaken professionalism.

7.1.3 Rule B (proposed): judges-only gating $\rightarrow$ fan-aware decision within the risk set

Our proposed institutional change is a *two-stage gate-and-decide* mechanism that makes professionalism structural: judges alone define who is at risk, and fan influence is applied only within that performance-defined boundary.

1. **Judges-only gate (risk set).** Rank contestants by judges’ performance $q_{i,t}^J$ and let B_t^J be the Bottom-2 under judges (Judge Bottom-2).

2. **Fan-aware decision within the gate.** Within B_t^J , compute $S_{i,t}$ and eliminate the contestant with the lower combined score:

$$\tilde{E}_{s,t} = \arg \min_{i \in B_t^J} (w_J q_{i,t}^J + (1 - w_J) p_{i,t}).$$

Why Rule B improves professionalism. By construction, elimination is always decided among the lowest-performing contestants according to judges. This prevents extreme popularity from overriding performance signals outside the judges-defined bottom tail, thereby strengthening professionalism, especially in early weeks when performance dispersion is largest.

Maintaining watchability without randomness: phase-dependent structured suspense. Because DWTS features celebrity participants, early-season skill gaps can be large and a strict professionalism-first gate may reduce perceived audience impact. We therefore treat watchability as a constrained secondary goal and introduce *structured suspense* only in late stages, while maintaining performance grounding:

- **Late-stage bottom- k band (within judges' tail).** When the field is small (e.g., remaining contestants $\leq K$), expand the judges-defined risk set from Bottom-2 to Bottom- k , and decide elimination within that band using $S_{i,t}$. This widens the decision boundary and increases suspense without leaving the low-performance region.
- **Limited structured multi-elimination (optional).** In a small number of late-stage weeks, allow multi-elimination, but restrict eliminations to the judges-defined bottom band. This increases stakes while remaining performance-grounded.

The phase boundary (K or a week threshold t_0) and the band size k are interpretable management knobs.

7.1.4 Stress testing via replay

What kind of stress test we employed to evaluate Rule B and select parameters?

Parameter sweep (control knobs).

Evaluation metrics (professionalism-first). We report:

- **Professionalism proxy: judges-tail alignment.** The eliminated contestant's weekly judges' rank/quantile distribution (greater concentration in the lower tail indicates higher professionalism).
- **Replay consistency (KPI1).** Week-level elimination-set agreement between observed and replayed outcomes (Jaccard mean under our stated convention).
- **Structured suspense proxy (KPI3 / FlipRate).** The rate at which elimination outcomes change across rules/parameters, capturing how institutional design introduces suspense.
- **Audience influence proxy.** Sensitivity of elimination outcomes to perturbations of $p_{i,t}$ within the feasible uncertainty range from Task A.

Robustness checks. We conduct those robustness checks:

In summary, Rule B makes professionalism a structural property through judges-only gating, while late-stage structured mechanisms preserve watchability without introducing randomness. Replay-based stress testing provides an interpretable policy region rather than a single tuned setting.

8 Conclusion

In this study, we model *Dancing with the Stars* as an inverse problem in which judges' scores and eliminations are observed while audience vote totals are withheld. Task A provides a convex inference framework to reconstruct weekly fan vote shares consistent with observed eliminations and to quantify identifiability and uncertainty. Building on these inferred votes, Tasks B and C implement season-week replays that isolate the impact of aggregation mechanics (percent versus rank) and optional bottom-two judges' save, revealing where rule changes most strongly affect outcomes. When a rule change keeps an observed-exit contestant active, we extend the replay using a conservative, minimal-information imputation solely for counterfactual completion, not as a performance forecast. Task D1 then separates the drivers of judges' evaluations and fan support using mixed-effects structures that attribute variation to celebrity attributes and professional partners. Finally, Task D2 proposes a simple, interpretable rule that power-transforms fan vote shares before percent blending, and evaluates it through stress tests over parameters and vote-uncertainty perturbations.

Collectively, our pipeline produces an auditable chain from data to mechanism interpretation to actionable rule recommendations, while explicitly flagging weeks and seasons where non-identifiability limits the strength of conclusions. Future work could incorporate richer behavioral signals (e.g., external popularity proxies) and explore alternative mechanism designs under the same replay-and-stress-test framework.

9 Memo to Management

To: Show Management

Subject: Recommendation for a More Stable and Transparent Voting System

Our analyses indicate that controversy risk concentrates in weeks where judges' evaluations and audience preferences diverge. To preserve audience agency while preventing extreme popularity from overwhelming technical performance, we recommend piloting *Judge Safeguard + Smoothed Rank (JS-SR)*.

Under JS-SR, each week designates a *Safeguard Zone*: contestants above a judges-score percentile threshold (top τ) are immune from elimination. The remaining contestants compete in a "battleground" where elimination is determined by a transparent weighted blend of judges' share and fan share:

$$S_{i,t}^{(JS)} = w_J q_{i,t}^J + (1 - w_J) p_{i,t}.$$

Management controls two intuitive knobs: τ (how many top technical performers are protected) and w_J (judge-fan balance within the battleground). We recommend selecting (τ, w_J) from a robust region identified by historical replays and uncertainty stress tests, then deploying JS-SR in a one-season "shadow mode" computed in parallel with the broadcast rule to quantify differences in eliminations, finalists, and winner outcomes before adoption.

Operationally, we recommend a one-season *shadow deployment* in which the proposed rule is computed in parallel with the broadcast rule to quantify differences in eliminations, finalists, and winner outcomes, accompanied by a clear public explanation of tie-handling and the role of any judges' save mechanism. This provides a controlled, auditable path to improving fairness perception and reducing outcome volatility without sacrificing interpretability.

References

- [1] COMAP, "2026 MCM problem C dataset (problem_c_data.csv)," MCM/ICM official contest materials, 2026, accessed: 2026-01-31. [Online]. Available: https://www.contest.comap.com/undergraduate/contests/mcm/contests/2026/problems/2026_MCM_Problem_C_Data.csv

- [2] D. G. Saari, *Decisions and Elections: Explaining the Unexpected*. Cambridge University Press, 2001. [Online]. Available: <https://www.cambridge.org/core/books/decisions-and-elections/>
- [3] COMAP, “2026 MCM problem C: Data with the stars (problem statement pdf),” MCM/ICM official contest materials, 2026, accessed: 2026-01-31. [Online]. Available: https://www.contest.comap.com/undergraduate/contests/mcm/contests/2026/problems/2026_MCM_Problem_C.pdf
- [4] S. Bubeck, “Convex optimization: Algorithms and complexity,” *Foundations and Trends in Machine Learning*, vol. 8, no. 3-4, pp. 231–357, 2015. [Online]. Available: <https://www.nowpublishers.com/article/Details/MAL-050>
- [5] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge University Press, 2004. [Online]. Available: <https://web.stanford.edu/~boyd/cvxbook/>
- [6] M. Benning and M. Burger, *Modern Regularization Methods for Inverse Problems*. Springer, 2018. [Online]. Available: <https://link.springer.com/book/10.1007/978-3-319-93978-0>
- [7] J. Kaipio and E. Somersalo, *Statistical and Computational Inverse Problems*. Springer, 2005. [Online]. Available: <https://link.springer.com/book/10.1007/b138659>
- [8] J. Aitchison, *The Statistical Analysis of Compositional Data*. Chapman and Hall, 1986.
- [9] V. Pawlowsky-Glahn and A. Buccianti, Eds., *Compositional Data Analysis: Theory and Applications*. Wiley, 2011. [Online]. Available: <https://onlinelibrary.wiley.com/doi/book/10.1002/9781119976462>
- [10] P. Filzmoser, K. Hron, and C. Reimann, “Compositional data analysis in practice,” in *Statistical Analysis of Compositional Data in the Geosciences*. Springer, 2018. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-319-96427-0_2
- [11] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, “Distributed optimization and statistical learning via the alternating direction method of multipliers,” *Foundations and Trends in Machine Learning*, vol. 3, no. 1, pp. 1–122, 2011. [Online]. Available: https://web.stanford.edu/~boyd/papers/admm_distr_stats.html
- [12] F. Brandt, V. Conitzer, U. Endriss, J. Lang, and A. D. Procaccia, Eds., *Handbook of Computational Social Choice*. Cambridge University Press, 2016. [Online]. Available: <https://www.cambridge.org/core/books/handbook-of-computational-social-choice/>
- [13] A. M. Law, *Simulation Modeling and Analysis*, 5th ed. McGraw-Hill Education, 2015.

A Appendix: Data, Algorithms, and Reproducibility

B Complete Files for Problem B

See [2625838](#) for repository tree and execution steps.

Report on Use of AI

Team Control Number: 2625838

Problem: C

AI Tools Used

AI Tool	Version / Model	Primary Purpose
OpenAI ChatGPT	GPT-5.2	Language polishing, structural refinement, and clarification of written explanations
OpenAI ChatGPT	GPT-5.2	Brainstorming ideas for modeling approaches, algorithm designs, and solution strategies
OpenAI ChatGPT	GPT-5.2	Generating preliminary code drafts for data processing, graph producing and simulation, later reviewed and modified by the team
OpenAI ChatGPT	GPT-5.2	Synthesis of the explanatory documents, work orders, template tickets for internal team Delivery
OpenAI ChatGPT	GPT-5.2	Giving Advice on engineered Document Management and Version Control best practices
Perplexity	Deep Research	Assistance on searching reference for relevant techniques
Github Copilot	—	Enhance productivity while maintaining full control over the code quality and integrity

Query/Response Record

ChatGPT Query 1:

Input: We just finished our setting of *dynamic_files*, please refer to the *first_stage_modeling.md* and *context_pack_for_ai.md* in our repository, and search for authoritative and highly credible supporting literature on topics of convex optimization/scalable solving, underdetermined inversion/regularization, ranking/rule differences, compositional data, simulation and sensitivity

Output: See full answers in [chatgpt_1_output.md](#).

ChatGPT Query 2:

Input: Please help us check our draft of our frame of the report as I listed, and give suggestions on improving the structure. (For the sake of clarity and format consistency, see the complete conversation record in [chatgpt_2_input_and_output.md](#))

Output: Your framework is usable, but it is recommended to revise Chapters 4–7 into a four-part structure: "Fan Vote Inference → Rule Replay Comparison → Impact Factors → New Mechanism Proposal". This way, the table of contents can directly correspond to the four task points of the problem at a glance, making it easier for reviewers to read and more like "answering the problem". (see the complete conversation record in [chatgpt_2_input_and_output.md](#))

ChatGPT Query 3:

Input: Please follow the work order ticket to finish the triage stage of data cleaning suggestions. (For the sake of clarity and format consistency, see the complete conversation record in [chatgpt_3_input_and_output.md](#))

Output: The season-week-contestant weekly panel has been successfully built in accordance with the P0 standards of *dynamic_files/data_agent_triage_ticket.md*, and the specified files have been delivered (including reproducible BL-0 baseline and consistency KPIs). See the complete conversation record in [chatgpt_3_input_and_output.md](#).

ChatGPT Query 4:

Input: Wait, why is the generated .pdf file completely unchanged? Could there be an issue with how we implemented the rules? Also, where in the code you provided do we need to modify the paths for importing and exporting files? (See the complete conversation record in [chatgpt_4_input_and_output.md](#))

Output: The most common reason you see "no changes" in the PDF is not an error in the rules, but that the path/file you're opening is not the newly generated one. Additionally, our BL-0 baseline inherently results in a FlipRate very close to 0, making the plot appear "largely unchanged". The plot you uploaded itself shows FlipRate fluctuating around 0 (with a very small coordinate range)—this is an expected outcome under BL-0. (see the complete conversation record in [chatgpt_4_input_and_output.md](#))

ChatGPT Query 5:

Input: Currently, our .py code files are located at 2026_MCM/src/LI_play_data/*.py, and we expect the generated charts to be saved to 2026_MCM/output/fig (or 2026_MCM/output/tab for tables, and the required data to be saved to 2026_MCM/output/data_cleaned). I have several questions:

- (a) Do we need to manually create the output/fig and output/tab directories in advance?
- (b) If not, do we need to modify the paths in the Python files, and how? (We do not plan to use .ipynb files.)
- (c) After modifying the paths in the files, how should I run the script on a Windows computer to generate the new files? Will the newly generated files overwrite the existing ones after running the script?

Output: Understood: You currently place the entry script at 2026_MCM/src/LI_play_data/*.py and expect outputs to be saved to:

- 2026_MCM/output/fig for figures
- 2026_MCM/output/tab for tables
- 2026_MCM/output/data_cleaned for cleaned data

Below are step-by-step answers to your 4 questions, along with a minimal modification approach (only adjust paths/directory creation without changing core logic). (see the complete conversation record in [chatgpt_5_input_and_output.md](#))

ChatGPT Query 6:

Input: You are the "Data Phase Master Controller + Standard Auditor" for my MCM/ICM project. Our problem-solving approach focuses on optimization/strategy/simulation, and we will not conduct general EDA. Please work only based on the files I upload; do not fabricate any data, results, fields, units, or problem requirements. If uncertain, mark it as "Not Provided/To Be Confirmed" and provide minimal supplementary information requirements (≥ 3 items) along with the safest default handling plan. (See complete prompt in [chatgpt_6_input_and_output.md](#))

Output: See full outputs in [chatgpt_6_output.md](#).

ChatGPT Query 7:

Input: What should we do regarding the previous conversation? Because the investigation file is non-authoritative; the files we feed to the agent do not include the investigation—it is only used for us to check if our framing/spec agent has any important omissions. Now clearly tell me how to use the information you provided above (e.g., 3. Place the Data Agent Triage work order in... file / place it separately in the conversation as a prompt).

Output: In line with your current constraints (the investigation file is non-authoritative and will not be fed to the Data Agent; it is only used to check for omissions in the framing/spec), you can use my previous output as two types of materials: (1) an executable work order for the Data Agent + (2) minimal standard patches for backfilling dynamic files. The specific operations are as follows (step by step, no need to modify the investigation file further). (see the complete conversation record in [chatgpt_7_input_and_output.md](#))

ChatGPT Query 8:

Input: 'Handling of "Withdrew": Should it be "exclude after inferring the exit week" or "exclude the entire season from the consistency denominator"? Consistency denominator: Exclude no-elimination weeks, use set-match for multi-elimination weeks (is this acceptable?).' What is the best way to handle this? Is it part of our cleaning standards? If we only add this one-sentence patch, will it cause huge problems in subsequent data processing?

Output: This is part of the cleaning standards, and it must be implemented in the kpi_registry; otherwise, two types of "huge problems" will occur later:

1. During the Data phase, everyone will handle withdrew/no-elimination/multi-elimination according to their own intuition, leading to inconsistent KPI denominators → irreproducible results;
2. In your writing, you mention "consistency", but when reviewers ask "how is the denominator calculated", you cannot answer or give inconsistent responses.

Therefore: Adding a patch is not a risk; on the contrary, it reduces risk. The real risk is "not clarifying it clearly". (see the complete conversation record in [chatgpt_8_input_and_output.md](#))

ChatGPT Query 9:

Input: Please check the setting of kpis from you and me and give me suggestions on how to deal with that. (See the complete conversation record in [chatgpt_9_input_and_output.md](#))

Output: When you said "I removed many KPIs", the reason is straightforward: I was defining the P0/Triage metrics framework—only retaining metrics that are 100% implementable with the core research brief + existing official CSV data, runnable within 2–4 hours, and require no data fabrication due to missing information. Several metrics in your original framework fall under the P1/P2 categories (requiring vote estimation models, additional assumptions, external data, or threshold calibration). To prevent the data team from being forced to "guess/impute/fabricate" data during the P0 phase, I temporarily excluded them from the minimal version of the authoritative KPI table. (see the complete conversation record in [chatgpt_9_input_and_output.md](#))

ChatGPT Query 10:

Input: Now let's go through the materials and prompt to be handed over to the Data Agent for triage: ... (see the complete conversation record in [chatgpt_10_input_and_output.md](#)) Could you check if there are any issues with this prompt?

Output: Your prompt is on the right track overall (with clear authoritative standards, ticket-driven execution and product delivery paths), but I recommend making 3 minor revisions; otherwise, the Data Agent may still make mistakes or deviate from the requirements during implementation: (see the complete conversation record in [chatgpt_10_input_and_output.md](#))

ChatGPT Query 11:

Input: Please refer to [chatgpt_11_input_and_output.md](#) for complete conversation record.

Output: See full outputs in [chatgpt_11_input_and_output.md](#)

ChatGPT Query 12:

Input: Our team members have completed the following work on the data: ... (see the complete conversation record in [chatgpt_12_input_and_output.md](#)) I think it may not be wise to continue spending time on this data, and our team member's modeling approach has already been finalized. Should we first complete the in-depth data cleaning and then compare with our team members to select the optimal approach, or directly proceed to the core work of our writing team after aligning our dynamic documents?

Output: Your intuition is correct: it is unwise to continue spending time on "general feature engineering/general cleaning". Now

that your core modeling approach is finalized (QP inversion for fan share + rule playback simulation + mixed effects), the optimal workflow is: First align and freeze the "specifications and data contract" → immediately integrate Model V1 to complete a closed-loop run → then conduct "model-driven deep cleaning/risk diagnosis (Top-3 risks)" → finally implement visualization and writing. Reason: Your model essentially relies on consistent specifications for "active set/ground truth eliminations/within-week normalization (percents)/N/A weeks and post-elimination zero-score encoding"; these are explicitly problematic areas in the problem statement (N/A weeks, zero scores, no/multiple eliminations, uncertainty about the return to rank-based aggregation in Season 28). (see the complete conversation record in [chatgpt_12_input_and_output.md](#))

ChatGPT Query 13:

Input: So let me sort out what we need to do now: First step: Align our specifications. You mentioned freezing and sticking to our core data pipeline, but actually I don't think this is necessary. Because after my teammate makes modifications, they will put their work in their own folder on GitHub, and mine will go into my folder. Even though we're in the same GitHub repository, we can cross-check each other's work later—it'll be like mutual supplementation. Second step: Integrating Model V1. I don't quite understand this part—does V1 refer to a code file (not a DOCX document) that will make the model we've designed actually run? Is that the right understanding? Then third step: Conduct in-depth data cleaning, followed by visualization and finalizing the writing. Is this the correct workflow? I also have another question: Earlier, we identified two risks of inconsistent specifications, right? Well, I suggest we fix these specification inconsistencies first—preferably by modifying the code. I'll send you the code, and could you guide me on exactly what needs to be changed?

Output: Your summary is mostly correct. Let me "calibrate" the key points, then directly guide you on **modifying the code first to unify specifications** (I've already reviewed the 'p0_triage_build_weekly_panel.py' file you sent this time). (see complete conversation record in [chatgpt_13_input_and_output.md](#))

ChatGPT Query 14:

Input: No rush, let's first sort out what we currently have:

1. We have the complete problem statement.
2. We have the preliminary data produced by our data team member just now.
3. We have the preliminary model developed by our modeling team member.

Please let me know what other files are needed and I will provide them.

Therefore, I think our workflow should be as follows:

1. First, complete the construction of the framework.
2. Then, provide you with our modeling spec, and then see which parts we can complete (my initial idea is that we can first finish the "Problem Restatement", "Key Assumptions", "Core Variables", and a bit of the beginning of the model; please supplement anything that is missing).
3. We will investigate the existing files and write the parts that can be included in the article.

Output: Your workflow plan is correct, and given that you "will not perform additional data cleaning in this phase", this is the most stable and rework-free way to proceed.

Below, I will refine your proposed 3 steps into an "executable checklist + the minimum set of files you need to provide + the writing blocks I can produce". By following this, you can directly start producing the main text.(see complete conversation record in [chatgpt_14_input_and_output.md](#))

Human Contribution Declaration

We affirm that:

1. All final modeling decisions, algorithm designs, and strategic choices were made by the team members.
2. All mathematical formulations and assumptions were independently derived or verified by the team.
3. All code included in this project was reviewed, tested, and fully understood by the team.
4. AI tools were used only for brainstorming, drafting, language refinement, and preliminary coding assistance.
5. The submitted work represents our own understanding and intellectual contribution.

Date: Feb 2, 2026